

标题

使用 # 号表示标题级别，数量从1到6。#+空格+标题文本的这种形式来书写。
语法示例：

```
# 这是一级标题
## 这是二级标题
### 这是三级标题
#### 这是四级标题
##### 这是五级标题
##### 这是六级标题
```

渲染结果：

这是一级标题

这是二级标题

这是三级标题

这是四级标题

这是五级标题

这是六级标题

加粗

使用两个 * 或 _ 包裹文字（这里的包裹是必须紧挨着的，且前后都有）(共4个符号)。其中_为下划线。
但是我们通常使用两个 * 进行包裹，然而对于两个_时，可能因为不同的编译器，在出现非加粗文字的时候，需要把非加粗文字和__中间加上空格。
语法示例：

```
这是**加粗文字**。  
这也是 __加粗文字__ 。
```

渲染结果：
这是**加粗文字**。
这也是 **加粗文字**。

斜体

使用一个 * 或 _ 包裹文字。但是推荐使用 *。这里类似于上面的加粗，由于编译器的不同，对于使用 _ 时，需要在不加粗内容和加粗内容之间加上空格。
语法示例：

这是 *斜体文字*。
这也是 _斜体文字_。

渲染结果：
这是*斜体文字*。
这也是 *斜体文字*。

列表

有序列表

使用数字加英文句点和空格。
语法示例：

- 1. 第一项
- 2. 第二项
- 3. 第三项

渲染结果：

- 1. 第一项
- 2. 第二项
- 3. 第三项

无序列表

使用 -、+ 或 * 加空格。

语法示例：

- 无序列表项
- + 无序列表项
- * 无序列表项

渲染结果：

- 无序列表项
- 无序列表项
- 无序列表项

定义列表

Markdown 支持定义列表，使用 : 来表示定义项。是一种用于展示术语（Term）及其对应定义（Definition）的列表形式，其核心作用是对特定名词或概念进行解释说明。

语法示例：

- 术语一
: 这是术语一的定义。
- 术语二
: 这是术语二的定义。

渲染结果：

- 术语一
: 这是术语一的定义。

术语二

这是术语二的定义。

待办事项列表（任务列表）

使用 - [] 表示未完成，- [x] 表示完成。

语法示例：

- [] 待办事项一
- [x] 已完成事项

渲染结果：

- ☐ 待办事项一
- ☒ 已完成事项

列表嵌套

无论是无序列表还是有序列表，创建子列表的关键都是在子列表项前进行缩进。常见的缩进方式是使用2个、4个空格或一个制表符（Tab）。使用快捷键Shift+Tab可以取消列表(这个快捷键一般是编辑器的功能)。

无序列表的嵌套

语法示例：

- 一级列表项 A
 - 二级列表项 A.1
 - 二级列表项 A.2
- 一级列表项 B
 - 二级列表项 B.1
 - 三级列表项 B.1.a
 - 三级列表项 B.1.b

渲染结果：

- 一级列表项 A
 - 二级列表项 A.1
 - 二级列表项 A.2
- 一级列表项 B
 - 二级列表项 B.1
 - 三级列表项 B.1.a
 - 三级列表项 B.1.b

有序列表的嵌套

语法示例：

1. 第一步骤
 1. 子步骤一
 2. 子步骤二
2. 第二步骤
 1. 子步骤 A
 1. 更详细的步骤 i
 2. 更详细的步骤 ii

渲染结果：

1. 第一步骤
 - i. 子步骤一
 - ii. 子步骤二
2. 第二步骤
 - i. 子步骤 A
 - a. 更详细的步骤 i
 - b. 更详细的步骤 ii

混合嵌套：有序与无序列表结合

Markdown支持有序列表和无序列表的混合嵌套，这提供了极大的灵活性。

语法示例：

- 项目准备
 1. 确定需求
 2. 准备资源
- 开发阶段
 - 前端开发
 1. 编写HTML结构
 2. 添加CSS样式
 - 后端开发

渲染结果：

- 项目准备
 - i. 确定需求
 - ii. 准备资源
- 开发阶段
 - 前端开发
 - a. 编写HTML结构
 - b. 添加CSS样式

- 后端开发

在列表中嵌套其他区块

列表项内部还可以嵌套其他Markdown元素，如引用区块（Blockquotes）或代码块。这需要在嵌套的内容前添加额外的缩进（通常是4个空格或一个制表符）。

语法示例：

- * 今日任务
 - 完成项目周报的撰写，并同步给团队成员。
- * 明日计划

渲染结果：

- 今日任务
 - 完成项目周报的撰写，并同步给团队成员。
- 明日计划

代码及代码块（灰色底框）

行内代码

使用单个反引号```包裹。左右各一个```紧贴。

语法示例：

这是一个 ``行内代码`` 的例子。

渲染结果：

这是一个 `行内代码` 的例子。

代码块（围栏代码块）

使用三个反引号`````包裹(共6个```)，并可指定语言。

语法示例：

```
```python
def hello():
 print("Hello, World!")
```
```

渲染结果：

```
def hello():
    print("Hello, World!")
```

分割线

使用三个或更多的 *、- 或 _。

语法示例：

```
***
---
_
```

渲染结果：

删除线

使用两个波浪号 ~~ 包裹文字（共四个~）。

语法示例：

这是一段 ~~被删除的~~ 文字。

渲染结果：

这是一段 被删除的 文字。

高亮

使用双等号 == 包裹文本。这是最直观的Markdown扩展语法，但并非所有平台都支持。这种方式在公众号编辑器中是不支持的，因此下图是我电脑编译器的渲染结果。Markdown 原生不支持高亮，但是有些编辑器是支持的。除了这种方式，还有HTML的方式进行高亮，这将是Markdown进阶语法知识。语法示例：

这是一个==高亮==。

渲染结果：

这是一个高亮。

链接

行内式链接 (Inline Link)

这是最直接的方式，链接的文本和地址写在一起。

普通链接（无标题）

语法示例：

[链接文字](URL)

[访问我的文章](https://mp.weixin.qq.com/s/fuDeBgUb5Xnt69jA4egPWA)

渲染结果：

[链接文字](#)

[访问我的文章](https://mp.weixin.qq.com/s/fuDeBgUb5Xnt69jA4egPWA)

带标题的链接：在URL后添加用双引号包裹的标题文本，鼠标悬停时会显示。

语法示例：

[访问我的文章](https://mp.weixin.qq.com/s/fuDeBgUb5Xnt69jA4egPWA "研途漫漫得偿所愿的文章")

渲染结果：

[访问我的文章](https://mp.weixin.qq.com/s/fuDeBgUb5Xnt69jA4egPWA)

自动链接 (Automatic Link)

对于标准的网址或邮箱地址，直接用尖括号 < > 包裹，Markdown会自动将其转换为可点击的链接。这种方式不能自定义显示文本。

语法示例：

欢迎访问我们的官网：<<https://markdown.com.cn>>

欢迎投稿：<yantumanman@163.com>

渲染结果：

欢迎访问我们的官网：<https://markdown.com.cn>

欢迎投稿：yantumanman@163.com

引用式链接 (Reference Link)

这种链接分为两部分：使用部分在正文中，定义部分可以在文档任意位置（通常在文末）。这能让正文更简洁，也便于统一管理链接。

基本用法

语法示例：

我推荐阅读 [GitHub官方教程][gh-docs]。

你可以在 [GitHub][gh]上创建账户。

[gh-docs]: <https://docs.github.com> "GitHub官方文档"

[gh]: <https://github.com>

渲染结果：

我推荐阅读 [GitHub官方教程](#)。

你可以在 [GitHub](#)上创建账户。

隐式链接标记

如果标识符为空，则会使用链接文字本身作为标识符进行匹配。

语法示例：

这个项目托管在[GitHub][]上。

[GitHub]: <https://github.com>

渲染结果：

这个项目托管在[GitHub](#)上。

如何选择链接方式（通常）：

行内式链接：适合短文档、博客文章或链接数量较少（如少于5个）的场景，写作流畅，一目了然。

引用式链接：适合长文档、技术文档或需要多次使用同一链接的场景。它能保持正文可读性，并且只需在一处修改链接地址即可全局更新，便于维护。

自动链接：适合快速插入不需要自定义文本的原始URL或邮箱地址。

引用

使用 > 符号。

语法示例：

- > 这是一个引用块。
- > 这是引用的第二行。

渲染结果：

这是一个引用块。
这是引用的第二行。

图片

在Markdown中插入图片的语法为： `![替代文本](图片地址或路径 "可选标题")`。其中的地址或路径主要分为两种：

- 相对路径：相对于当前Markdown文件的位置。例如，当图片与文档在同一文件夹时，可直接使用 `![关注我](图片a研途漫漫.jpg)`。“图片a研途漫漫”是我为这张图片的命名。
- 绝对路径：从系统根目录开始的完整路径。例如，如果图片存放在 `D:\Markdown\` 目录下，可以使用 `![关注我](D:/Markdown/图片a研途漫漫.jpg)`。但是绝对路径只能在我的这台特定电脑的D盘这个位置工作。
- 然而，使用本地路径（无论是相对还是绝对）存储图片时，在跨设备同步上会遇到困难。为了解决这个问题，最佳的实践是使用图床——即将图片上传至云端服务器，生成一个永久的网络链接。其语法格式为： `![示例图片](https://example.com/image.jpg)`。只要设备能够联网，Markdown文档

在任何地方都能正确显示图片，完美实现了文档与图片的分离与同步。

语法示例：

![关注我](图片a研途漫漫.jpg)

![关注我](D:/Markdown/图片a研途漫漫.jpg)



研途漫漫得偿所愿

微信扫描二维码，关注我的帐号



研途漫漫得偿所愿

微信扫描二维码，关注我的帐号

表格

使用管道符 | 分隔单元格，用 - 分隔表头，并用 : 控制对齐。可以使用冒号 : 指定对齐。:--- 左对齐（默认）、:---: 居中对齐、---: 右对齐

语法示例：

```
左对齐	居中对齐	右对齐
单元格1	单元格2	单元格3
单元格4	单元格5	单元格6
```

渲染结果：

| 左对齐 | 居中对齐 | 右对齐 |
|--------|--------|--------|
| 单元格1 | 单元格2 | 单元格3 |
| 这是单元格4 | 这是单元格5 | 这是单元格6 |

脚注

在文中用 [^标识符] 标注，在文末用 [^标识符]: 定义内容。

语法示例：

```
这是带脚注的文本[^1]。  
  
[^1]: 这是脚注内容。  
  
Markdown是一种轻量级标记语言[^note1]，广泛应用于文档编写[^note2]。  
  
[^note1]: 由约翰·格鲁伯（John Gruber）和亚伦·斯沃茨（Aaron Swartz）共同创建。  
[^note2]: 例如，技术博客、项目README文件以及学术笔记等[^note3]。  
[^note3]: 许多静态网站生成器，如Hugo、Jekyll，都原生支持Markdown。
```

渲染结果：

这是带脚注的文本^[1]。

Markdown是一种轻量级标记语言^[2]，广泛应用于文档编写^[3]。

带外框的文本

这通常不是标准Markdown语法，这是LaTeX数学公式的 `\boxed{}` 命令（需在支持数学公式的环境中使用）。在之后的Markdown进阶语法中会介绍LaTeX数学公式，这里之所以会讲，是因为这个还是很常见的，尤其是Deepseek给出最终结果的时候经常出现。

语法示例：

```
$\boxed{文本}$
```

渲染结果：



实体字符与转义

在Markdown中直接书写某些特殊字符（如<, >, &）可能会被解析，如需原样显示，需使用HTML实体或转义。在之后的Markdown进阶语法中也会详细介绍html。

1.实体字符:

- 小于号：< 或 \<
- 大于号：> 或 \>
- 和号：& 或 \&

渲染结果：

- 小于号：< 或 <
- 大于号：> 或 >
- 和号：& 或 &

常见的转义

一些字符具有特殊意义，如果需要显示它们本身，通常需要使用反斜杠进行转义，这在很多编程语言中都是有的。

使用反斜杠 \ 可以转义以下具有特殊含义的字符： \ ` * _ { } [] () # + - . ! ~ | > ^ \$ % & = @ 。例如： * 显示为 *， \# 显示为 #， \[显示为 [。

写在最后

希望这份整理好的语法能帮助您系统地学习Markdown。请注意，本文介绍的部分高级功能（如带外框的文本、复杂表格）的渲染效果可能因编辑器或平台的支持程度而异。

1. 这是脚注内容。↩
2. 由约翰·格鲁伯（John Gruber）和亚伦·斯沃茨（Aaron Swartz）共同创建。↩
3. 例如，技术博客、项目README文件以及学术笔记等^[4]。↩
4. 许多静态网站生成器，如Hugo、Jekyll，都原生支持Markdown。↩